INTRODUCTION TO HASH CRYPTOGRAPHY

Subject: Cryptography and Network Security

Assignment

Q1. Define hash cryptography. Explain the fundamental principles and techniques of cryptographic hash functions with suitable examples.

Hash cryptography is the use of mathematical hash functions to convert data of any size into a fixed-size value called a **hash value**, **message digest** or **fingerprint**. A cryptographic hash function is designed to be easy to calculate but extremely difficult to reverse or manipulate in a useful way.

Fundamental principles

- **Fixed-length output:** Whatever the size of the input, the hash function produces an output of a fixed length. For example, SHA-256 always produces a 256-bit hash.
- **Deterministic:** The same input always produces the same hash value.
- **One-way operation:** It should be practically impossible to recover the original message from its hash.
- **Avalanche effect:** A tiny change in the input should produce a very different output.
- **Collision resistance:** It should be very difficult to find two different inputs that produce the same hash.

How it works

The message is given as input to a hash algorithm. The algorithm processes the data and returns a fixed-length digest. The digest can then be stored or compared to check whether the data has changed.

```
Message / File → Cryptographic Hash Function → Fixed-length Hash (Digest)
```

Example: If a user downloads a file, the website may provide its SHA-256 hash. The user calculates the hash of the downloaded file. If both values match, this gives strong evidence that the file has not changed during transfer.

Common hash algorithms include MD5, SHA-1, SHA-256 and SHA-3. MD5 and SHA-1 are no longer considered secure against collision attacks, so modern systems generally prefer SHA-256 or SHA-3.

Q2. Discuss the role of cryptographic hash functions in cryptographic applications. Explain their use in data integrity verification, digital signatures, password storage, and message authentication.

Cryptographic hash functions are important because they create a short fingerprint of data. This fingerprint is used in several security applications.

1. Data integrity verification

A hash can help check whether a file or message has been changed. The sender and receiver calculate the hash of the data and compare the results. Matching hashes indicate that the data is likely unchanged.

2. Digital signatures

In a digital signature system, the sender usually hashes the message first and then signs the digest using a private key. The receiver verifies the signature using the sender's public key and checks the message digest.
This helps provide integrity, authentication and non-repudiation.

3. Password storage

Websites should not store passwords as plain text. Instead, they store a password hash created with a password-hashing method such as Argon2id, bcrypt, scrypt or PBKDF2. A unique random salt is added to each password before hashing. During login, the entered password is processed and checked against the stored value.

4. Message authentication

A keyed-hash message authentication code (HMAC) combines a secret key with a hash function. The receiver uses the same secret key to verify the message and its authentication tag. HMAC helps detect message modification and confirm that the sender knows the shared key.

```
Message → Hash → Digest → Digital Signature (using private key)
```

Example: A banking application can use HMAC to check that a payment request has not been altered while travelling between the application and server.

Q3. Explain the advantages and disadvantages of hash cryptography. Discuss the major properties of a secure cryptographic hash function, including preimage resistance, second-preimage resistance, and collision resistance.

Hash cryptography is useful for protecting and verifying information, but it also has limitations.

Advantages

- Produces a short, fixed-size fingerprint for data of any length.
- Helps detect accidental or intentional changes to files and messages.
- Works efficiently in digital signatures, authentication and integrity checking.
- A secure hash is designed to be one-way, so the original input is not directly revealed.

Disadvantages

- A hash alone does not encrypt data and cannot be used to recover the original message.
- Different inputs can theoretically produce the same hash; this is called a collision.
- Fast general-purpose hashes are unsuitable for directly storing passwords because attackers can testguesses quickly.
- Weak or outdated algorithms, such as MD5 and SHA-1, can be attacked and should not be used wherecollision resistance is required.

Three major security properties

- **Preimage resistance:** Given a hash value, it should be computationally infeasible to find an input that produces it.
- **Second-preimage resistance:** Given one specific input, it should be infeasible to find a different input with the same hash.
- **Collision resistance:** It should be infeasible to find any two different inputs that produce the same hash.

Difference: Preimage resistance starts with a hash value; second-preimage resistance starts with a particular message; collision resistance allows the attacker to choose both messages. A secure cryptographic hash function aims to provide all three properties.

Q4. Describe the Secure Hash Algorithm (SHA) family. Explain the development and key features of SHA-1, SHA-2, and SHA-3, highlighting their major differences.

The **Secure Hash Algorithm (SHA)** family is a group of cryptographic hash algorithms standardized for security applications. Different versions were developed over time to improve security and meet new requirements.

1. SHA-1

SHA-1 was published in the 1990s and produces a 160-bit hash value. It was widely used in digital certificates and other systems. However, practical collision attacks have been demonstrated, so SHA-1 is now considered insecure for applications requiring collision resistance.

2. SHA-2

SHA-2 was designed as a stronger successor to SHA-1. It includes SHA-224, SHA-256, SHA-384 and SHA-512. For example, SHA-256 produces a 256-bit digest. SHA-2 is widely used in digital signatures, certificates and secure communication.

3. SHA-3

SHA-3 was standardized in 2015 after a public competition. It uses a different internal design called the Keccak sponge construction, unlike the compression-function design used by SHA-1 and SHA-2. SHA-3 includes SHA3-224, SHA3-256, SHA3-384 and SHA3-512.

Major differences

```
Algorithm Output examples Design / current status
SHA-1 160 bits Older design; collision-broken; avoid
SHA-2 224, 256, 384, 512 Widely used; considered secure
SHA-3 224, 256, 384, 512 Sponge construction; alternative to SHA-2
```

Summary: SHA-1 is outdated, while SHA-2 and SHA-3 are modern secure choices. SHA-3 provides a different design, which gives systems an alternative if weaknesses are ever found in SHA-2.

Q5. Explain the applications of SHA algorithms in cryptographic protocols and information security. Discuss their role in digital certificates, digital signatures, file integrity checking, and secure communication.

SHA algorithms are used to create message digests that help protect information. SHA-2 and SHA-3 are commonly preferred for modern security applications.

1. Digital certificates

Digital certificates bind an identity, such as a website name, to a public key. Certificate authorities digitally sign certificate information using a signature algorithm that typically hashes the certificate data first. This helps browsers verify that the certificate has not been modified and was issued by a trusted authority.

2. Digital signatures

SHA algorithms create a digest of a document or message. The digest is signed with the sender's private key. The receiver verifies the signature and compares the digest to check the document's integrity and origin.

3. File integrity checking

A SHA-256 checksum can be published beside a software download. After downloading, users calculate the file's SHA-256 hash and compare it with the published checksum. A match helps show that the file is unchanged, provided the checksum itself came from a trusted source.

4. Secure communication

Protocols such as TLS use cryptographic hash functions in authentication and integrity mechanisms. HMAC can protect messages, while modern TLS cipher suites may use authenticated encryption. SHA-256 and SHA-384 are also used in certificate signatures and other protocol operations.

```
File / Message → SHA-256 → Digest → Compare or use in a signature / HMAC
```

Example: When a user visits an HTTPS website, certificates and cryptographic operations help verify the server's identity and protect communication. Hash algorithms are one part of this overall security process.

Conclusion: SHA algorithms support integrity and authentication in many security systems. They do not encrypt information by themselves; they work alongside encryption, digital signatures and authentication mechanisms.
